Reviewer Pack — Loomis-Whitney Marginal Defect of 3-XOR Solution Lattice Tra…

Entry 17460b6d540e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 22:35:08 UTC

Loomis-Whitney Marginal Defect of 3-XOR Solution Lattice Tracks SoS Refutation Threshold

Entry ID: `17460b6d540e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 22:35:08 UTC

1. Conjecture

Field A (mathematical branch): Loomis-Whitney / Bollobás-Thomason coordinate-projection inequalities on the discrete cube (combinatorial measure theory; Loomis-Whitney 1949, Bollobás-Thomason 1995, uniform cover inequalities) **Field B** (complexity object): Sum-of-Squares degree-2 pseudo-feasibility / refutation threshold for random 3-XOR CSPs

Statement:

> For a 3-XOR instance φ with m = αn clauses on n Boolean variables, let S_τ(φ) := {x ∈ {0,1}^n : x satisfies ≥ τm clauses of φ} and let π_i: {0,1}^n → {0,1}^{n-1} drop coordinate i. Define the Loomis-Whitney marginal defect LW(φ,τ) := (1/(n-1)) · Σ_{i=1}^{n} log_2 |π_i(S_τ(φ))| − log_2 |S_τ(φ)|, which is ≥ 0 by Loomis-Whitney. The conjecture: at τ = 1 − 1/(8α), the rescaled defect LW(φ,τ)/n converges in probability to a function ψ(α) that is monotone non-decreasing, equals 0 for α ≤ α_sat ≈ 0.918 (the random-3-XOR satisfiability/degree-2 SoS feasibility threshold), and is strictly positive for α > α_sat; equivalently, for every random 3-XOR instance with α ≤ 0.85 we have LW(φ,τ)/n < 0.02, while at α ≥ 1.10 we have LW(φ,τ)/n > 0.05 with probability ≥ 0.9.

Rationale (proposer's reasoning):

> The degree-2 SoS pseudo-distribution is a relaxation of the *uniform* distribution on near-satisfying assignments, so its existence is constrained by how 'product-like' the true satisfying set is across coordinate projections — exactly what Loomis-Whitney quantifies. Below the SAT threshold the solution lattice is essentially a full-dimensional union of orbits (Loomis-Whitney is tight on Hamming-thick sets), while above it the surviving near-satisfying assignments are forced to lie in a low-codimension affine variety whose marginal projections shrink faster than the set itself — a quantitative analogue of the freezing/condensation transition. This bridge avoids Natural Proofs because LW is a relational marginal quantity, not a single-number truth-table invariant, and it is not poly-time computable in n.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e029575f2d63b326`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.84	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from itertools import product

def evaluate_instance(instance, assignment):
    return sum(assignment[i] ^ instance[i][j] for i in range(len(instance)) for j in range(len(ins

def compute_LW(instance, alpha):
    n = len(instance)
    tau = 1 - 1 / (8 * alpha)
    S_tau = {tuple(sorted([assignment[i] if i != j else 1 - assignment[j] for j in range(n)]))
             for assignment in product([0, 1], repeat=n)
             if evaluate_instance(instance, assignment) >= tau * len(instance)}

    total_size = len(S_tau)
    marginal_sizes = [len({tuple(sorted([x[i] if i != j else 1 - x[j] for j in range(n)])) for x i

    LW = sum(math.log2(size) for size in marginal_sizes) / n - math.log2(total_size)
    return LW

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [14, 16, 18, 20]
    alpha_values = [0.5, 0.7, 0.85, 0.918, 1.00, 1.10, 1.50]

    results = []
    for n in n_values:
```

```
        for alpha in alpha_values:
            instance = [[random.randint(0, 1) for _ in range(n)] for _ in range(alpha * n)]
            LW = compute_LW(instance, alpha)
            results.append(LW / n)

    mean_LW = sum(results) / len(results)
    std_LW = (sum((x - mean_LW) ** 2 for x in results) / len(results)) ** 0.5

    conjecture_holds = all(0 <= LW < 0.02 for LW in results if alpha <= 0.85) and \
                       all(LW > 0.05 for LW in results if alpha >= 1.10)

    return {
        "metric_name": "LW/alpha",
        "metric_value": mean_LW,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean_LW = sum(results) / len(results)
    std_LW = (sum((x - mean_LW) ** 2 for x in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if 0 <= r < 0.02 and 0.85 <= alpha_values[results.in

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_LW} std={std_LW} support_fraction={support_fraction}"
    elif any(0.85 <= r < 0.02 for r in results):
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[res
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={suppor
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34e7de56.py", line 66, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34e7de56.py", line 42, in run_trial
    instance = [[random.randint(0, 1) for _ in range(n)] for _ in range(alpha * n)]
                                                                       ^^^^^^^^^^^^^^^^
TypeError: 'float' object cannot be interpreted as an integer
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to type error before producing data; cannot assess support/falsification | next: Fix the integer conversion error in instance generation (replace alpha*n with int(alpha*n) or similar)

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	371263
2	preregistration	ollama_remote	qwen3:8b	0	0	53187
3	novelty	ollama_remote	qwen3:8b	0	0	27336
4	novelty	ollama_remote	qwen3:8b	0	0	23273
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16220
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12322
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11842
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10254
9	judge	ollama_remote	qwen3:8b	0	0	24186

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 549884 ms total latency. Provider mix: {'claude_max': 1, 'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/17460b6d540e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/17460b6d540e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/17460b6d540e.tar.gz` (if generated)